

Lemon Aid — 데이터베이스 완전 가이드

비전공자도 처음부터 끝까지 이해할 수 있도록 작성한 DB 구조 설명서

담당: D (1

목차

1. 데이터베이스란 무엇인가?

2. 왜 Pos

1. 데이터베이스란 무엇인가?

1.1 일상 언어로 설명

앱을 쓸 때 입력하는 모든 정보는 어딘가에 저장되어야 합니다. 앱을 껐다 켜도 내 정보가 그대로 있는 이유가 바로 데이터베이스(DB)가 정보를 보관하기 때문입니다.

가장 쉬운 비유는 도서관입니다.

도서관 = 데이터베이스 전체

서가(책장) =

1.2 Excel과 비교

많은 분들이 익숙한 Excel로 비교하면 이렇습니다.

Excel	데이터베이스
-------	--------

파일(.xlsx)	데이터베이스
시트(탭)	테이블
행	레코드(record)
열	컬럼(column)
셀 서식	데이터 타입 (숫자, 문자, 날짜...)
시트 간 참조(=Sheet1!A1)	외래키(Foreign Key)

그렇다면 왜 Excel 대신 DB를 쓰나요?

Excel의 한계:

- 동시에 C

2. 왜 PostgreSQL인가?

2.1 데이터베이스 종류

세상에는 많은 종류의 데이터베이스가 있습니다. 크게 두 가지로 나뉩니다.

관계형 DB (테이블 구조) 비관계형 DB (자유 구조)

Oracle

2.2 왜 Oracle이 아닌가?

레몬헬스케어 실무에서는 Oracle을 사용합니다. 하지만 이 프로젝트에서 Oracle을 쓰지 않는 이유:

이유	설명
비용	Oracle은 CPU 코어당 연간 수백만 원
설치 복잡도	설치와 설정만 수일 소요
학생 환경	라이선스 없이 합법적으로 쓸 수 없음

Oracle Free(Express) 버전이 있지만 2GB RAM, 2 CPU 제한이 있어 OCR + LLM + 시계열 DB를 동시에 돌리기 어렵습니다.

2.3 왜 MySQL이 아닌가?

MySQL도 무료이지만 이 프로젝트에서 부족한 부분이 있습니다.

필요한 기능	MySQL	PostgreSQL
JSON 데이터 유연 저장	제한적	JSONB로 완벽 지원
시계열 확장(TimescaleDB)	없음	있음
행 단위 보안(RLS)	없음	있음

전문 검색(Full Text)	약함	강함
Oracle 문법 유사도	낮음	높음

2.4 PostgreSQL을 선택한 3가지 핵심 이유

이유 ①: JSONB — 영양제 성분표를 자유롭게 저장

영양제 제품마다 성분이 다릅니다. 비타민C만 있는 것도 있고, 30가지 성분이 있는 종합비타민도 있습니다. 이걸 고정된 열로 만들면:

일반 방식 (열 고정):

비타민C | 비

이유 ②: TimescaleDB — 걸음수·심박수를 빠르게 처리

사용자 한 명이 하루 1,440분 동안 매 분마다 걸음수를 기록하면 하루에 1,440건입니다. 사용자 1,000명이면 하루 144만 건입니다. 이 시계열 데이터를 일반 DB로 처리하면:

일반 PostgreSQL: 최근 30일 조회 → 약 0.8초

TimescaleD

TimescaleDB는 PostgreSQL 위에 얹는 플러그인이므로 추가 비용 없이 사용할 수 있습니다.

이유 ③: RLS — 내 정보는 나만 볼 수 있다

Row Level Security(행 단위 보안)는 PostgreSQL 내부에서 "본인 데이터만 볼 수 있게" 강제하는 기능입니다.

-- 이 설정 하나로, user_id가 다른 행은 절대 조회 불가

CREATE PO

3. 왜 Docker로 실행하는가?

3.1 Docker가 없다면

각 팀원이 자기 컴퓨터에 PostgreSQL을 직접 설치해야 합니다. 문제는:

팀원 A (Mac): PostgreSQL 16 설치 → 잘 됨

팀원 B (Win

3.2 Docker를 쓰면

`docker-compose up -d` ← 명령어 하나로 끝

Docker는 미리 만들어진 환경을 담은 상자(컨테이너)를 실행하는 도구입니다. 우리가 쓰는 상자 두 개:

컨테이너 이름	내용물	포트
---------	-----	----

lemon_aid_db	PostgreSQL 16 + TimescaleDB	5432
lemon_aid_redis	Redis 7	6379

4. 왜 Redis도 같이 쓰는가?

4.1 Redis란

Redis는 초고속 임시 저장소입니다. PostgreSQL이 도서관이라면 Redis는 포스트잇입니다.

PostgreSQL (도서관): 영구 저장, 느리지만 안전

Redis (포스트잇): 초고속 임시 저장

4.2 이 프로젝트에서 Redis 용도

용도	설명	효과
OCR 결과 캐시	같은 영양제 사진은 다시 분석 안 함	비용 50% 절감
KDRIs 룩업 캐시	영양소 권장량 테이블 (자주 읽힘)	DB 부하 감소
API Rate Limit	사용자당 분당 5회 제한	과도한 비용 방지

5. 외부 데이터 소스 3종 분석

이 프로젝트에서 사용하는 외부 공공 데이터 3종의 구조와 우리 DB에 적재하는 방식을 설명합니다.

5.1 AI Hub — 한국 음식 이미지 데이터셋

출처: <https://aihub.or.kr> (데이터셋 번호 79)

용도: 음식 이미지 분류

제공 데이터 구조

AI Hub 음식 이미지 데이터

— 이미지 파일명

우리 DB 매핑

AI Hub 컬럼 우리 DB 테이블.컬럼

이미지는 DB에 저장하지 않습니다. S3/NCP Object Storage에 올리고 URL(주소)만 저장합니다.

이미지를

5.2 식약처 식품영양정보 표준DB

출처: <https://data.mfds.go.kr>

용도: 음식

제공 데이터 구조

식약처 표준DB 컬럼 구조

— 기본정

우리 DB 매핑

식약처 컬럼 우리 DB 테이블.컬럼

5.3 식품안전나라 영양정보 DB

출처: <https://various.foodsafetykorea.go.kr/nutrient>

용도: 비타

제공 데이터 카테고리

식품안전나라 영양 카테고리

— 일반성

우리 DB 매핑

모든 영양소 수치 → `foods.nutrients_per_100g` (JSONB)

6. 전체 데이터 구조 설계

6.1 데이터 흐름 개요

| 공공 DB

6.2 음식 데이터 테이블

| 테이블:

6.3 영양제 데이터 테이블

| 테이블:

7. 병원 데이터 + 사용자 데이터 연결 구조

7.1 왜 병원 데이터가 필요한가?

일반 영양제 앱:

사용자 입

7.2 병원 데이터 테이블

| 테이블:

7.3 사용자 데이터 + 병원 데이터 연결 방식

① 회원가입 + 온보딩 (사용자가 직접 입력)

users 테이블

7.4 Kaggle 데이터 → 우리 DB 매핑

Kaggle 컬럼

우리 DB 테이블.컬럼

8. 음식·영양제 섭취 기록 구조

8.1 식단 기록 흐름

사진 촬영 → 분석 Agent (OCR+AI) → 사용자 확인 → 저장

8.2 식단 기록 테이블

| 테이블:

8.3 영양제 복용 기록 테이블

| 테이블:

8.4 분석 결과 저장

| 테이블:

9. 전체 테이블 구조

5.1 현재 구현된 테이블 ()

users — 회원 기본 정보

| 테이블명

profiles — 건강 프로필

refresh_tokens — 로그인 토큰 관리

consents — 개인정보 동의 이력

5.2 구현 예정 테이블

supplements (마스터) supplement_ingredients (성분 상세)

daily_scores — 하루 식단 점수

	user_id
--	---------

건강 데이터 — TimescaleDB Hypertable

step_counts (걸음수)	
-------------------	--

10. 테이블 간의 관계

모든 데이터는 users 테이블을 중심으로 연결됩니다.

	user_id
--	---------

관계 종류 설명

1:1 관계 (한 명에 하나)

users

11. 파일 구조와 역할

7.1 전체 파일 지도

backend/

|

7.2 각 파일의 역할 상세

config.py — 설정 읽기

.env 파일에서 값을 읽어 다른 파일들이 쓸 수 있게 제공

settings.dat

db/base.py — 모든 테이블의 공통 틀

class Base(DeclarativeBase):

pass # 1

db/session.py — DB 연결 통로

DB와 연결하는 문(세션)을 열고 닫는 역할

async def g

models/user.py — 테이블 설계도

class User(Base, TimestampMixin):

__tablename__

utils/security.py — 보안 도구

hash_password("1234") # "1234" → "\$2b\$12\$xyz..." (암호화)

verify_passw

utils/deps.py — 로그인 확인

API 호출 시 "이 사람 로그인한 사람 맞아?" 자동 확인

async def g

12. 요청부터 저장까지 — 전체 흐름

8.1 회원가입 흐름

[Flutter 앱]

|

8.2 로그인 흐름

[Flutter 앱]

|

8.3 프로필 저장 흐름

[Flutter 앱] ← 로그인 후 온보딩 화면에서

|

13. 보안 처리 방식

9.1 비밀번호 — bcrypt 해싱

비밀번호는 절대 원문으로 저장하지 않습니다. bcrypt 알고리즘으로 복호화가 불가능한 해시값으로 변환합니다.

사용자 입력: "mypassword123"

DB 저장:

9.2 JWT 토큰 — 신분증 시스템

로그인 후 매번 비밀번호를 보내는 것은 위험합니다. 대신 임시 신분증(JWT 토큰) 을 발급합니다.

JWT 토큰 구조:

eyJhbGciOiJ

토큰	유효시간	용도
access_token	30분	모든 API 호출 시 첨부, 짧아서 유출돼도 피해 최소화
refresh_token	7일	access_token 만료 시 재발급용, DB에 저장

토큰 사용 흐름:

로그인 →

9.3 민감정보 — AES-256 암호화 (예정)

만성질환, 복약정보는 의료법상 민감정보입니다. 현재는 JSONB로 저장 중이며, 추후 AES-256-GCM으로 암호화 예정입니다.

AES-256이란?

- 은행,군사

14. SQLAlchemy — Python과 DB를 연결하는 다리

10.1 왜 SQLAlchemy를 쓰나?

SQL을 직접 쓰면 이렇게 됩니다:

SQL 직접 작성 (문자열 오류 위험, SQL Injection 위험)

query = f'S

SQLAlchemy(ORM)를 쓰면:

Python 문법으로 작성 (타입 체크, 보안 자동 처리)

user = await

10.2 ORM이란?

ORM(Object-Relational Mapping)은 DB 테이블을 Python 클래스로 표현하는 방식입니다.

PostgreSQL 테이블 Python 클래스

users 테이블

15. Alembic — 테이블 변경 이력 관리

11.1 왜 필요한가?

개발 중에 테이블 구조가 바뀔 때마다 "나는 컬럼 추가했는데 다른 팀원 DB에는 없어서 오류남" 상황이 생깁니다. Alembic은 이런 변경을 버전으로 관리합니다.

Git이 코드 변경을 관리하듯, Alembic은 테이블 변경을 관리

11.2 사용 방법

1. 모델(Python 코드) 변경 후 마이그레이션 파일 자동 생성

alembic rev

16. 현재 구현된 API 전체 목록

http://localhost:8000/docs 에서 Swagger UI로 테스트 가능합니다.

경로	메서드	기능	인증 필요
/api/v1/auth/signup	POST	회원가입	
/api/v1/auth/login	POST	로그인 → JWT 발급	
/api/v1/auth/refresh	POST	access 토큰 갱신	
/api/v1/auth/logout	POST	로그아웃	
/api/v1/profile	GET	내 프로필 조회	Bearer
/api/v1/profile	PUT	내 프로필 저장/수정	Bearer

Swagger에서 테스트하는 방법

1. http://localhost:8000/docs 접속

2. POST /au

17. 로컬 실행 순서

13.1 처음 시작할 때 (최초 1회)

1. Docker 설치 (<https://docs.docker.com/get-docker/>)

13.2 이후 매번 개발할 때

`docker-compose up -d` # DB 실행 (꺼져 있을 때)

uvicorn src.

18. 자주 하는 실수와 해결법

오류 메시지	원인	해결
Connection refused	Docker DB가 안 켜져 있음	<code>docker-compose up -d</code>
relation does not exist	마이그레이션 안 함	<code>alembic upgrade head</code>
401 Unauthorized	토큰 없거나 만료됨	재로그인 → 새 토큰 발급
422 Unprocessable	요청 데이터 형식 오류	요청 Body 확인
duplicate key value	이미 가입된 이메일	다른 이메일로 시도
ModuleNotFoundError	<code>pip install</code> 안 함	<code>pip install -r requirements.txt</code>

19. 다음 구현 예정 항목

순서	항목	필요한 테이블	데이터 출처
1	음식 데이터 적재	foods	식약처 Open API (공공데이터포털)
2	영양제 데이터 적재	supplements, supplement_ingredients	식약처 건강기능식품 DB
3	KDRIs 데이터 적재	data/kdris_2020.csv	한국영양학회 2020
4	식단 기록 저장	meals	사용자 입력 + OCR
5	영양제 복용 기록	user_supplements	사용자 입력
6	분석 결과 저장	diagnoses, daily_scores	AI Agent
7	건강 데이터 수집	step_counts, weight_logs	HealthKit / Health Connect
8	AES-256 암호화	profiles (만성질환, 복약)	내부 구현

9	이메일 인증	email_verifications	SMTP / AWS SES
---	--------	---------------------	----------------